

 Member-only story

Using `pymupdf4llm`: From Simple PDF Extraction to Embedding AI-Generated Image Descriptions

6 min read · Feb 12, 2026



Martin Keywood

Follow



Listen



Share

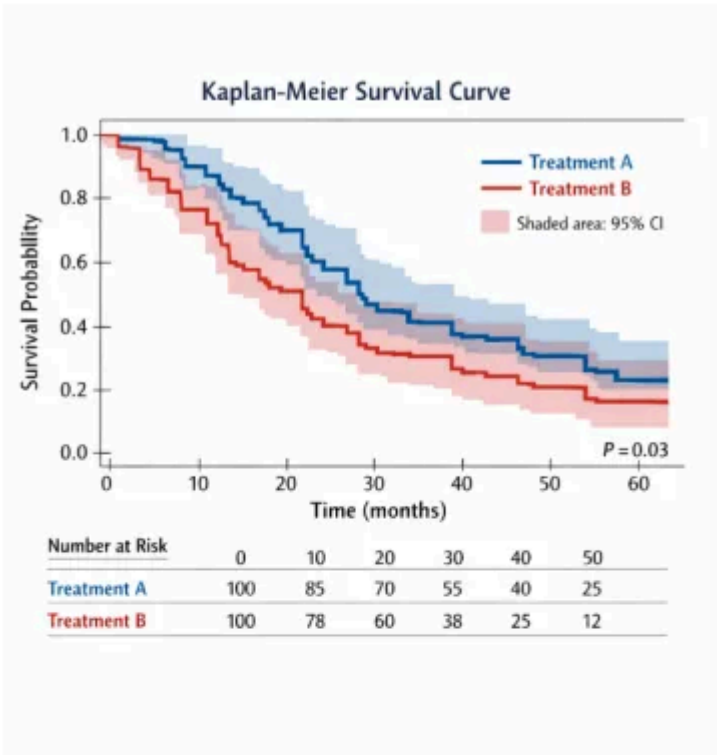


More

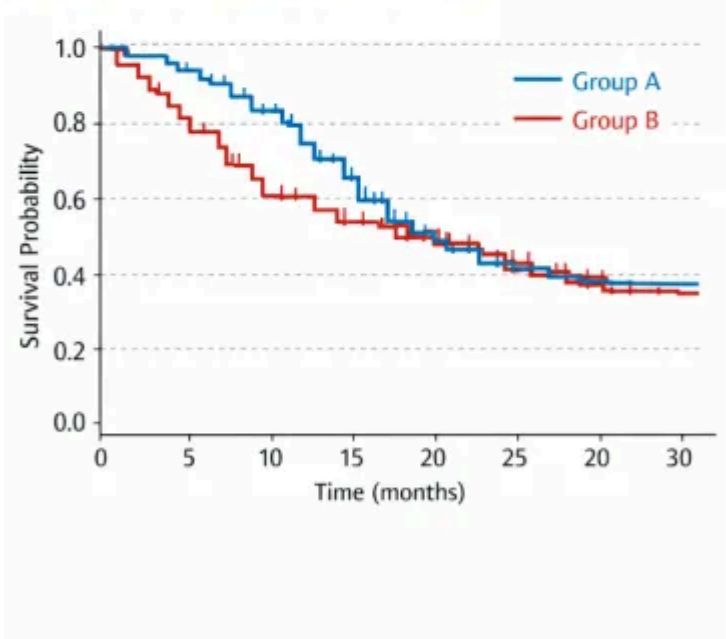
I love those moments when a small weekend experiment spirals into something unexpectedly delightful — the kind where you start with “I wonder if I can parse this PDF...” and end up building an automated workflow that extracts text, extracts images, generates descriptions for those images using a vision model, and finally embeds everything back into clean Markdown for use as context for downstream LLM usage.

That’s exactly what happened when I started playing with `pymupdf4llm` and a test document I created, `survival_plots_with_titanic_table.pdf`, containing two Kaplan-Meier survival plots and a sample Titanic dataset table. It looks like this:

Kaplan-Meier Curve 1
This figure shows survival probability trends for two treatments.



Kaplan-Meier Curve 2
Another comparison of survival patterns between two groups.



Titanic Sample Data Table

Passenger	Class	Age	Sex	Survived
John Doe	3rd	22	Male	No
Mary Smith	1st	38	Female	Yes
Karl Jensen	2nd	27	Male	Yes
Anna Petrova	3rd	15	Female	Yes

Of course we've all done extractions of PDF, probably using PyMuPDF, to extract PDF text and use with LLM's, but if you haven't used it yet, `pymupdf4llm` is a clever layer built on top of PyMuPDF that focuses on one thing: **converting complex PDFs into beautifully structured Markdown, with optional image extraction, embedding, and layout reconstruction**. It's powerful — and surprisingly flexible when mixed with modern LLMs.

Today's walkthrough:

- Installing and using `pymupdf4llm`
- Extracting text
- Extracting images
- Embedding images directly in Markdown
- Chunking pages
- Fixing the ONNX int32 → int64 issue
- And finally: **automatically generating image descriptions and embedding them back into the Markdown**

Installing `pymupdf4llm`

It can be installed simply by:

```
pip install -U pymupdf4llm pymupdf4llm[ocr,layout]
```

Step 1 — Straightforward PDF → Markdown Conversion

I started by converting the PDF.

Here's the basic script:

```
import pymupdf4llm
md_text = pymupdf4llm.to_markdown("survival_plots_with_titanic_table.pdf")
```

```
import pathlib
pathlib.Path("output.md").write_bytes(md_text.encode())
```

This gives you clean text, figure captions, table titles — really nice, but no images yet.

Step 2 — Enabling Layout Handling (and Fixing ONNX)

To get realistic block structures and tables, I enabled the layout model:

```
import numpy as np
import pymupdf.layout # activate PyMuPDF-Layout
import pymupdf4llm
import pathlib
import pymupdf.layout
import pymupdf4llm

# Monkey-patch numpy to force int64 for ONNX compatibility
_original_array = np.array
def patched_array(*args, **kwargs):
    result = _original_array(*args, **kwargs)
    if result.dtype == np.int32:
        result = result.astype(np.int64)
    return result
np.array = patched_array

# now work with the markdown text, e.g. store as a UTF8-encoded file
md_text = pymupdf4llm.to_markdown("survival_plots_with_titanic_table.pdf")
pathlib.Path("output_layout.md").write_bytes(md_text.encode())
```

Now you get recognisable figure blocks and table structure.

NOTE: It includes monkey patching Numpy to get around a horrible ONNX int32 / int64 bug I was getting at runtime. All it's basically doing is converting int32 to int64.

Step 3 — Extracting Images

Because the document contains two survival curves and a Titanic dataset table, I wanted the images exported to disk:

```
md_text = pymupdf4llm.to_markdown("survival_plots_with_titanic_table.pdf",
                                  write_images=True,
```

```
image_path="images_output")
pathlib.Path("output_layout_with_images.md").write_bytes(md_text.encode())
```

This populates `images_output/` with PNGs for each extracted figure. Nice !!

Step 4 — Embedding Images Directly in Markdown

For completeness, I also tried embedding base64 images:

```
md_text = pymupdf4llm.to_markdown("survival_plots_with_titanic_table.pdf",
                                  embed_images=True)
pathlib.Path("output_layout_with_embedded_images.md").write_bytes(md_text.encode())
```

So this put the Base64 of the image directly into the Markdown. Great for portability — terrible for Git diffs. But a lovely, simple and convenient feature.

Step 5 — Chunking Pages

Chunking splits the document into manageable model-friendly text segments:

```
chunks = pymupdf4llm.to_markdown("survival_plots_with_titanic_table.pdf",
                                  page_chunks=True)

print(len(chunks))

print(chunks[0]["text"])

print(chunks[1]["text"])
```

Useful for RAG pipelines, but not strictly necessary for this workflow.

Step 6 — Full Pipeline: Extract Text → Extract Images → Describe Images → Embed Descriptions

This is the fun part.

I built a script that:

1. Converts the PDF to Markdown

2. Extracts images
3. Summarize the document **before** image descriptions (using Gemma3 via Ollama)
4. Feeds each image to a vision model (using LLaVA via Ollama)
5. Embeds those descriptions back into the Markdown generated in step 3
6. Summarizes again **after** description embedding (using Gemma3 via Ollama)
7. Saves everything

```
import re
import pathlib
from pathlib import Path
import numpy as np
import pymupdf.layout # activate PyMuPDF-Layout with ONNX int64 fix
import pymupdf4llm
import ollama

# Monkey-patch numpy to force int64 for ONNX compatibility
_original_array = np.array
def patched_array(*args, **kwargs):
    result = _original_array(*args, **kwargs)
    if result.dtype == np.int32:
        result = result.astype(np.int64)
    return result
np.array = patched_array

# Parse without interpreting images first
md_text = pymupdf4llm.to_markdown("survival_plots_with_titanic_table.pdf",
                                write_images=True,
                                image_path="Images_output")

# Summarize the markdown with Gemma 3
summary_before = ollama.chat(
    model="gemma3",
    messages=[
        {
            "role": "user",
            "content": "Summarize the following markdown content:\n\n" + md_text
        }
    ],
)

print("Summary (before image descriptions):")
print(summary_before["message"]["content"])
```

```

# Extract image filenames from markdown
markdown_image_pattern = re.compile(r"![[^\]]*\]\(((^)+)\)")
html_image_pattern = re.compile(r"<img[^>]*src=\"([^\"]+)\"[^\>]*>", re.IGNORECASE)

image_paths = []
for pattern in (markdown_image_pattern, html_image_pattern):
    for match in pattern.finditer(md_text):
        raw_path = match.group(1).strip()
        if raw_path.startswith("data:"):
            continue
        candidate = Path(raw_path)
        if not candidate.is_absolute():
            candidate = candidate
        image_paths.append(candidate)

# de-duplicate while preserving order
seen = set()
unique_image_paths = []
for p in image_paths:
    key = str(p)
    if key not in seen:
        seen.add(key)
        unique_image_paths.append(p)

# run vision model for each image and embed descriptions into markdown
image_descriptions = {}
for image_path in unique_image_paths:
    res = ollama.chat(
        model="llava:7b",
        messages=[
            {
                "role": "user",
                "content": "Describe this image:",
                "images": [str(image_path)],
            }
        ],
    )
    image_descriptions[str(image_path)] = res["message"]["content"].strip()

def embed_image_descriptions(markdown_text: str) -> str:
    def replace_md(match: re.Match) -> str:
        raw_path = match.group(1).strip()
        candidate = Path(raw_path)
        if not candidate.is_absolute():
            candidate = candidate
        desc = image_descriptions.get(str(candidate))
        if not desc:
            return match.group(0)
        return f"{match.group(0)}\n\n> Image description: {desc}\n"

```

```
def replace_html(match: re.Match) -> str:
    raw_path = match.group(1).strip()
    candidate = Path(raw_path)
    if not candidate.is_absolute():
        candidate = candidate
    desc = image_descriptions.get(str(candidate))
    if not desc:
        return match.group(0)
    return f"{match.group(0)}\n\n> Image description: {desc}\n"

updated = markdown_image_pattern.sub(replace_md, markdown_text)
updated = html_image_pattern.sub(replace_html, updated)
return updated
```

[Open in app ↗](#)



```
# Summarize again with Gemma 3
summary_after = ollama.chat(
    model="gemma3",
    messages=[
        {
            "role": "user",
            "content": "Summarize the following markdown content (including image descriptions)"
        }
    ],
)

print("\nSummary (after image descriptions):")
print(summary_after["message"]["content"])

# Now work with the markdown text, e.g. store as a UTF8-encoded file
pathlib.Path("embed_images.md").write_bytes(md_text.encode())

# Save updated markdown with embedded image descriptions
pathlib.Path("embed_images_with_descriptions.md").write_bytes(
    md_text_with_descriptions.encode()
)
```

Summary Before Adding Image Descriptions

Summary (before image descriptions):

This document presents two Kaplan-Meier survival curves comparing two treatments (likely based on the Titanic dataset). It includes a sample data table of passenger information (Passenger, Class, Age, Sex, and Survived) used in the

analysis. The figure captions indicate that the survival curves are visually represented in two separate images (linked above, presumably PDFs).

Summary After Embedding Model-Generated Image Descriptions

Summary (after image descriptions):

This document presents two Kaplan-Meier survival curves comparing two treatment groups, alongside a sample Titanic dataset table.

Kaplan-Meier Curve 1: This graph (image 1) illustrates survival probability trends for “Treatment A” and “Treatment B” over a period of 0–8 months. The blue line represents Treatment A, initially showing a high survival rate, which then decreases over time. The red line represents Treatment B, also displaying a high survival rate which then decreases over time. The graph includes shaded areas representing the 95% confidence intervals for both Treatments.

Kaplan-Meier Curve 2: This graph (image 2) compares survival patterns for “Group A” and “Group B” across a time range of approx 6 years. The x-axis represents time (months), and the y-axis represents “Survival probability.” The lines fluctuate, and cross at 18 months and again at around 27 months.

Titanic Dataset: A small sample dataset is included, representing passengers from the Titanic. The table contains columns for Passenger name, Class, Age, Sex, and Survival (Yes/No).

Overall: The document presents a comparative analysis of survival rates, likely within a research study or clinical trial, utilizing Kaplan-Meier curves alongside a sample dataset. It highlights the use of statistical measures such as confidence intervals to quantify uncertainty in the survival probability estimates.

As you can see, MUCH MUCH richer, using descriptions of the images (albeit still pretty poor as I am only using small models on a local CPU laptop — imagine how cool bigger models would be) embedded in it to augment and enrich the core extracted text, and making it strong context for downstream LLM usage.

Final Thoughts

The document ended up being a perfect test case.

Two survival curves.

A clean Titanic sample table.

Clear visual structure.

And enough variation to test image detection, extraction, and description generation.

The result is Markdown text that:

- is LLM-ready
- has human-readable descriptions
- includes both text and vision model intelligence
- and produces dramatically better summaries

Exactly what I was hoping for.

Although, in hindsight, I should not have called it ‘embedding the image description’ as the word ‘embedding’ already has some strong connotations in LLM land, but I hope you can forgive me just this once.

As always, I hope you enjoyed this and it inspires you to try things yourself.

Thanks for reading!

LLM

Llm Applications



Follow

Written by Martin Keywood

90 followers · 3 following

Data Science, Machine Learning and LLMs are hugely exciting, enabling better data understanding to see beyond the obvious & help make decisions & help patients.

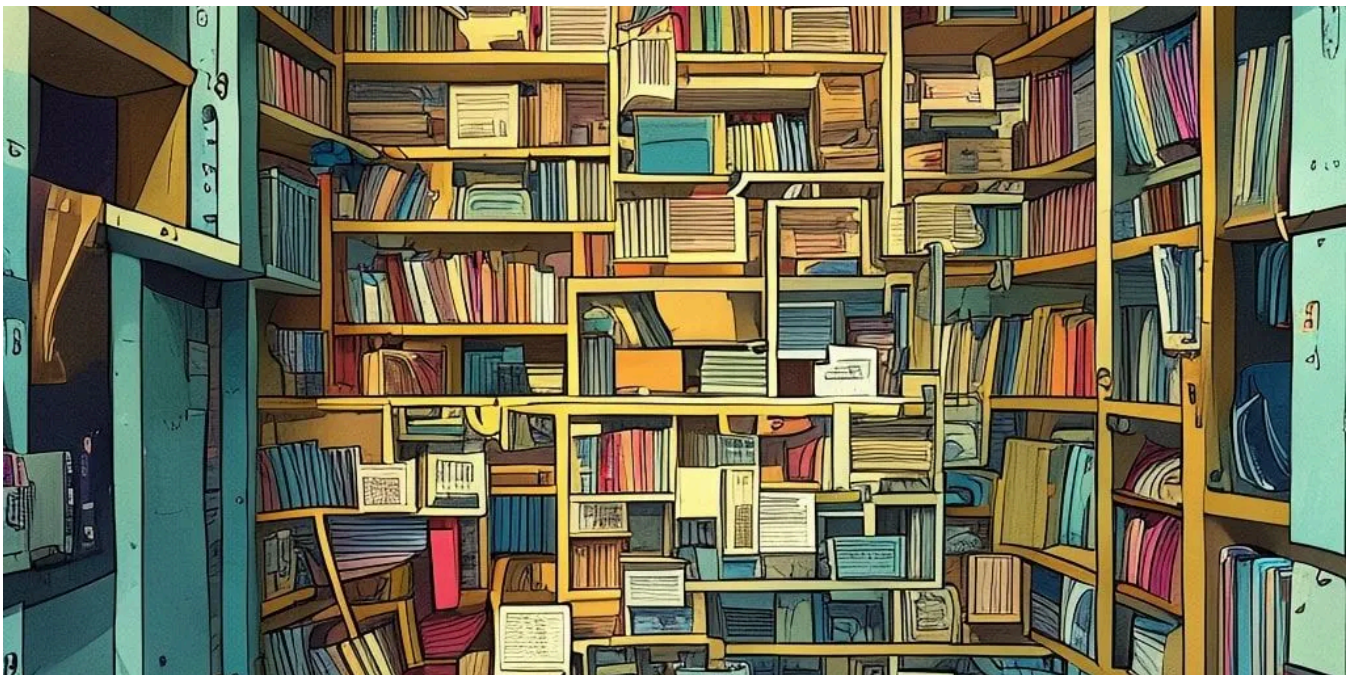
No responses yet



Stellars

What are your thoughts?

More from Martin Keywood



Martin Keywood

Can Language Models Actually Handle Long Documents? Recursive Language Models to the rescue!

What happens when you try to feed a language model more text than it can actually handle?

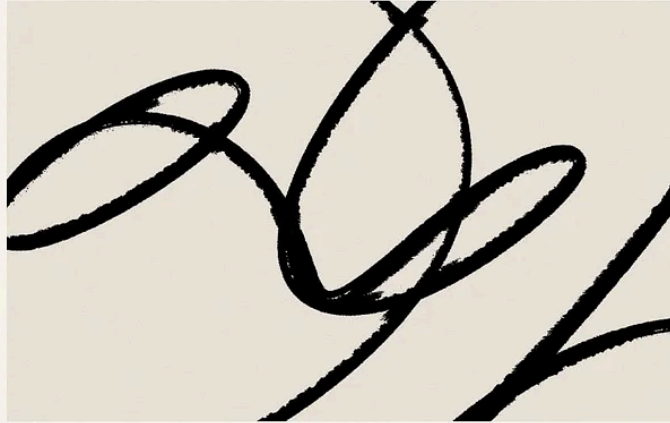


Jan 15



4





 Martin Keywood

Small Language Models: The Rise of SLMs—Are Tiny LMs Ready for Prime Time?

The Unexpected Journey into Small Language Models: My SLM Exploration

★ Aug 2, 2025 🖱 1



 Martin Keywood

Building My First MCP Tooling Stack: From Zero to Weather-and-Currency Agent (With Ollama)

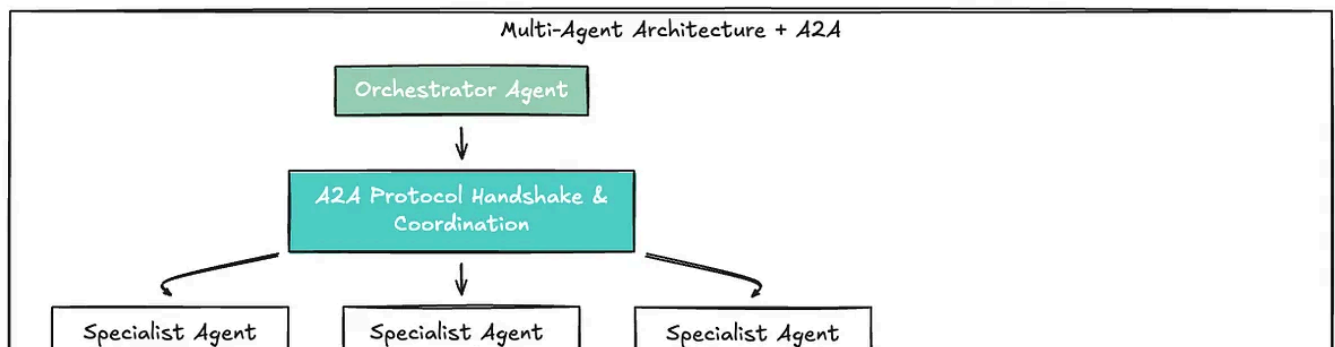
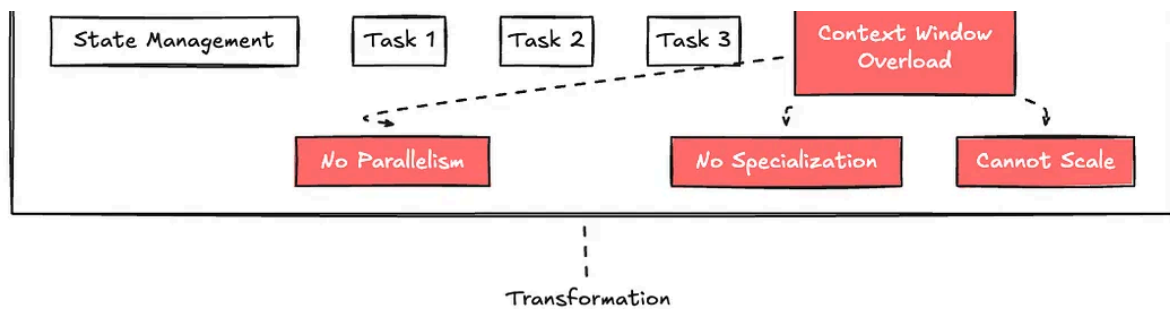
Why this? Honestly, I got curious. I keep seeing Model Context Protocol (MCP) popping up whenever folks talk about getting LLMs to use...

★ Oct 13, 2025



See all from Martin Keywood

Recommended from Medium



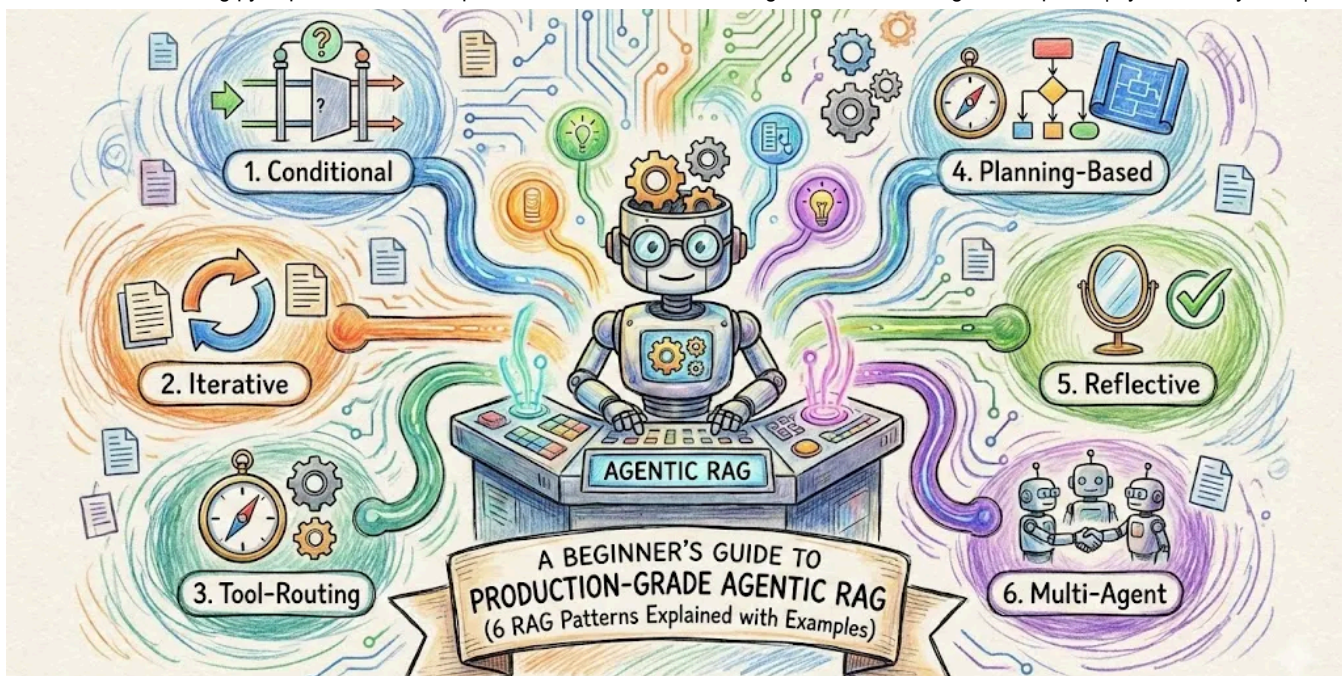
Yusuf Baykaloğlu

Multi-Agent Systems: Orchestrating AI Agents with A2A Protocol

A Deep Dive into Agent Architecture, Workflows, and Real-World Implementation

Jan 12 🖱 252





 In Towards AI by Divy Yadav

A Beginner's Guide to Production-Grade Agentic RAG (6 RAG Patterns Explained with Examples)

Six Agentic RAG patterns explained with real production trade-offs

★ Feb 10 🖱 294 💬 2

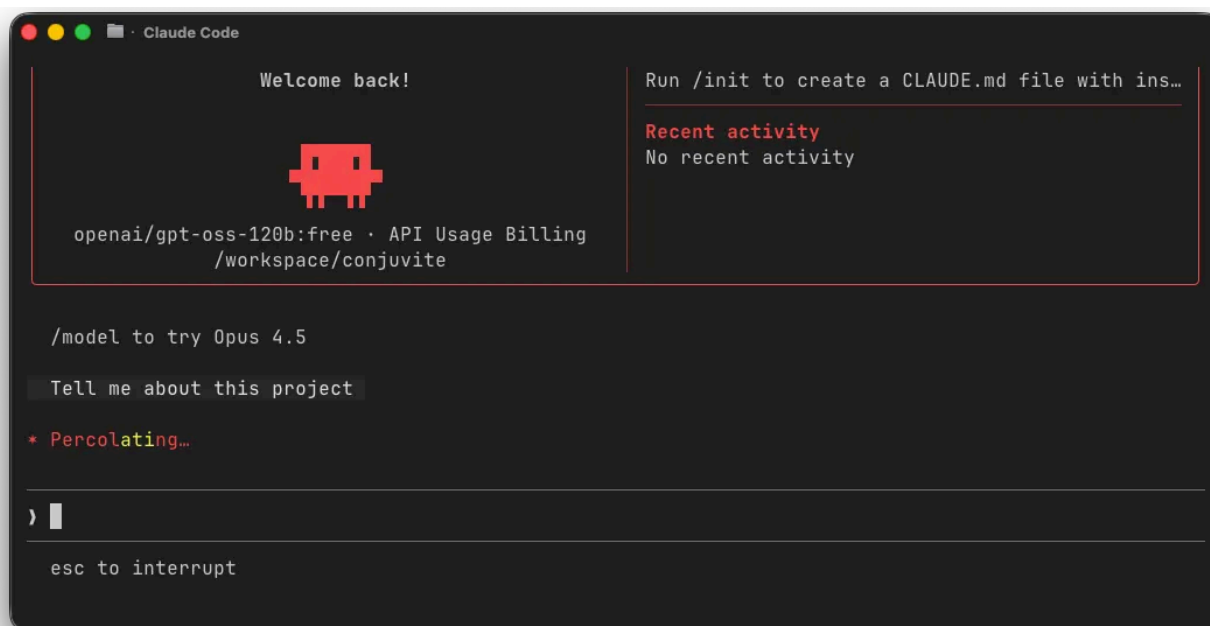


 cheng zhang

GLM-4.7-Flash: The Ultimate 2026 Guide to Local AI Coding Assistant

 Core Highlights (TL;DR)

Jan 20 🖱️ 95 💬 1



Luong NGUYEN

Run Claude Code with Local & Cloud Models in 5 Minutes (Ollama, LM Studio, llama.cpp, OpenRouter)

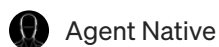
This post exists because the old guides were written back when using non-Anthropic models in Claude Code required hacks and weird adapters...

Jan 31 🖱️ 427 💬 13



In Level Up Coding by Christian Bernecker

From Theory to Implementation: Building a Robust, Self-Correcting AI Agent from Scratch with Python

Small team of engineers can easily burn >\$2K/mo on Anthropic's Claude Code (Sonnet/Opus 4.5). As budgets are tight, you might be wondering...

<https://medium.com/@martinkeywood/using-pymupdf4llm-from-simple-pdf-extraction-to-embedding-ai-generated-image-descriptions-b15d366f...> 16/16